

3-TIER BANKING WEB APPLICATION ON AWS Manual Deployment Using AWS Management Console

A Secure 3-Tier Banking Web Application Deployed Manually on AWS using NGINX, Flask/Gunicorn, Amazon EC2 and Amazon RDS

1. Aim of the Project

The aim of this project is to design and manually deploy a secure 3-tier banking web application on Amazon Web Services (AWS).

The infrastructure resources were created manually through the AWS Management Console. The application was also deployed manually on AWS EC2 instances without using Infrastructure as Code (IaC) tools such as Terraform or AWS CloudFormation.

The architecture separates the application into three logical tiers:

- **Web Tier** – NGINX
- **Application Tier** – Flask with Gunicorn
- **Database Tier** – Amazon RDS MySQL

The project demonstrates practical AWS cloud deployment, network configuration, security group implementation, EC2 server management, application deployment, reverse proxy configuration and database provisioning.

2. Project Overview

The project is a banking-style web application developed using Flask and deployed on AWS using a manually configured 3-tier architecture.

The application provides basic banking portal functionality such as:

- Landing page

- User signup
- User login
- Dashboard
- Logout

The application infrastructure was created manually using the AWS Management Console. Each AWS resource was configured individually according to the requirements of the application.

The application code was manually transferred/deployed to the EC2 Application Server, dependencies were installed, and the Flask application was configured to run using Gunicorn.

NGINX was manually installed and configured on the Web Server. It acts as a reverse proxy and forwards application requests to the Flask application running on the Application Server.

Amazon RDS MySQL was manually created as the Database Tier. The database is hosted inside the private network and is not directly accessible from the public internet.

3. Technologies Used

Layer	Technology	Purpose
Web Tier	NGINX	Reverse proxy and static file serving
Application Tier	Python / Flask	Web application framework
Application Server	Gunicorn	Runs the Flask application
Process Management	systemd	Manages the Gunicorn service
Database Tier	Amazon RDS MySQL	Relational database
Compute	Amazon EC2	Hosts application servers
Networking	Amazon VPC	Provides isolated cloud network
Load Balancing	Application Load Balancer	Distributes incoming application traffic
Security	AWS Security Groups	Controls network access
Operating System	Ubuntu Server 24.04 LTS	EC2 operating system
Source Code	Git / GitHub	Application source-code management
Deployment Method	Manual	AWS resources and application deployed manually

4. System Architecture

The application follows a traditional 3-tier architecture in which each tier performs a specific role.

Internet users access the application through the **Application Load Balancer (ALB)**. The ALB forwards incoming requests to the **NGINX Web Tier** hosted on an EC2 instance.

The NGINX Web Tier serves static content and acts as a reverse proxy. Dynamic requests are forwarded to the **Flask Application Tier**, which runs using Gunicorn on a separate EC2 instance.

The Flask Application Tier processes the application logic and communicates with **Amazon RDS MySQL** for database operations. User registration and other application data are written to the RDS MySQL database through the application.

Secure Server Access Flow

The private Web and Application EC2 instances are not accessed directly from the internet.

The `.pem` private key was used for secure SSH authentication when accessing the EC2 instances. The Bastion Host was used as the secure access point for reaching the private Web and Application servers.

SCREENSHOT PLACEMENT — ARCHITECTURE

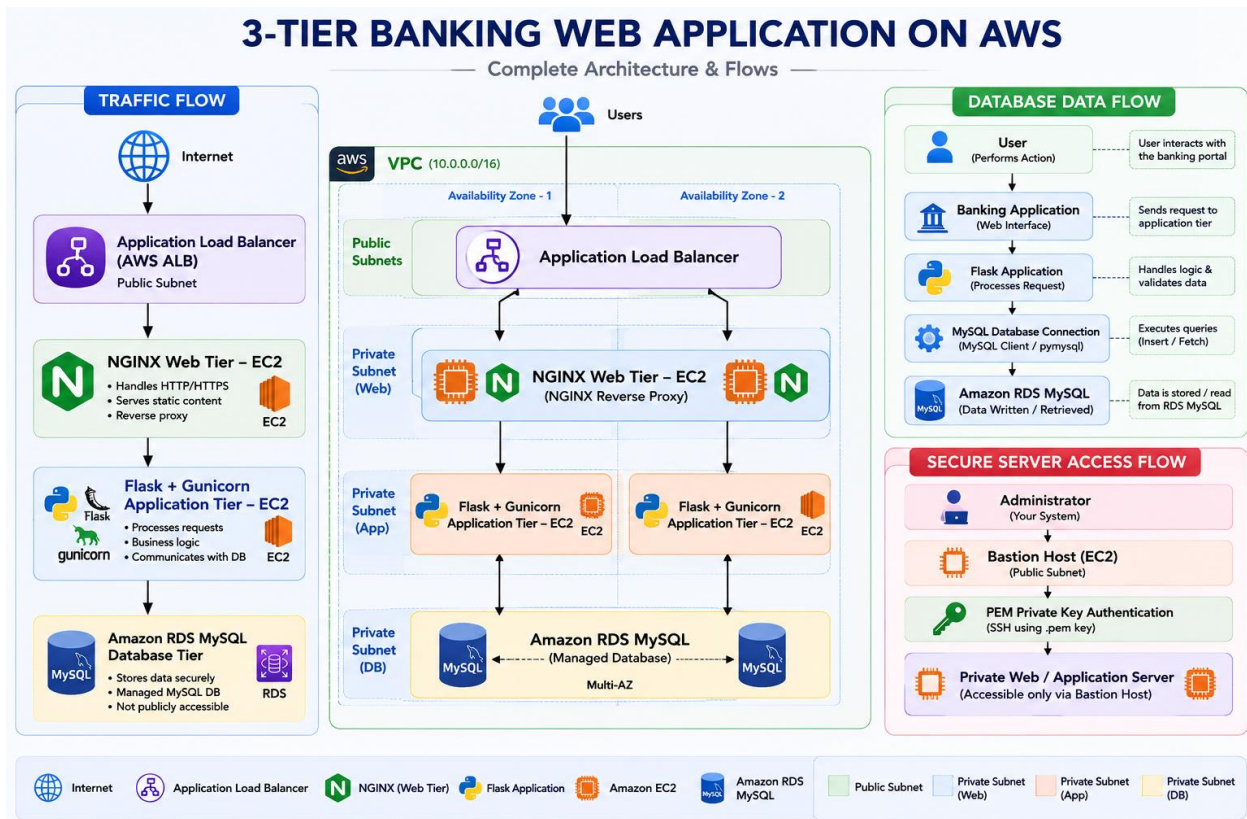


Fig. 1 — 3-Tier Banking Web Application Architecture showing Internet, ALB, NGINX Web Tier, Flask Application Tier and Amazon RDS MySQL

5. Network & Infrastructure Design

5.1 VPC and Subnet Configuration

A dedicated VPC was manually created using the AWS Management Console.

The VPC contains public and private subnets for separating the different application tiers.

Subnet	CIDR	Type	Purpose
pub-subnet	10.0.1.0/24	Public	ALB and Bastion Host

pub-subnet-2	10.0.6.0/24	Public	ALB
web-subnet	10.0.2.0/24	Private	NGINX Web Tier
app-subnet	10.0.3.0/24	Private	Flask Application Tier
db-subnet	10.0.4.0/24	Private	RDS MySQL
db-subnet-2	10.0.5.0/24	Private	RDS MySQL

SCREENSHOT PLACEMENT

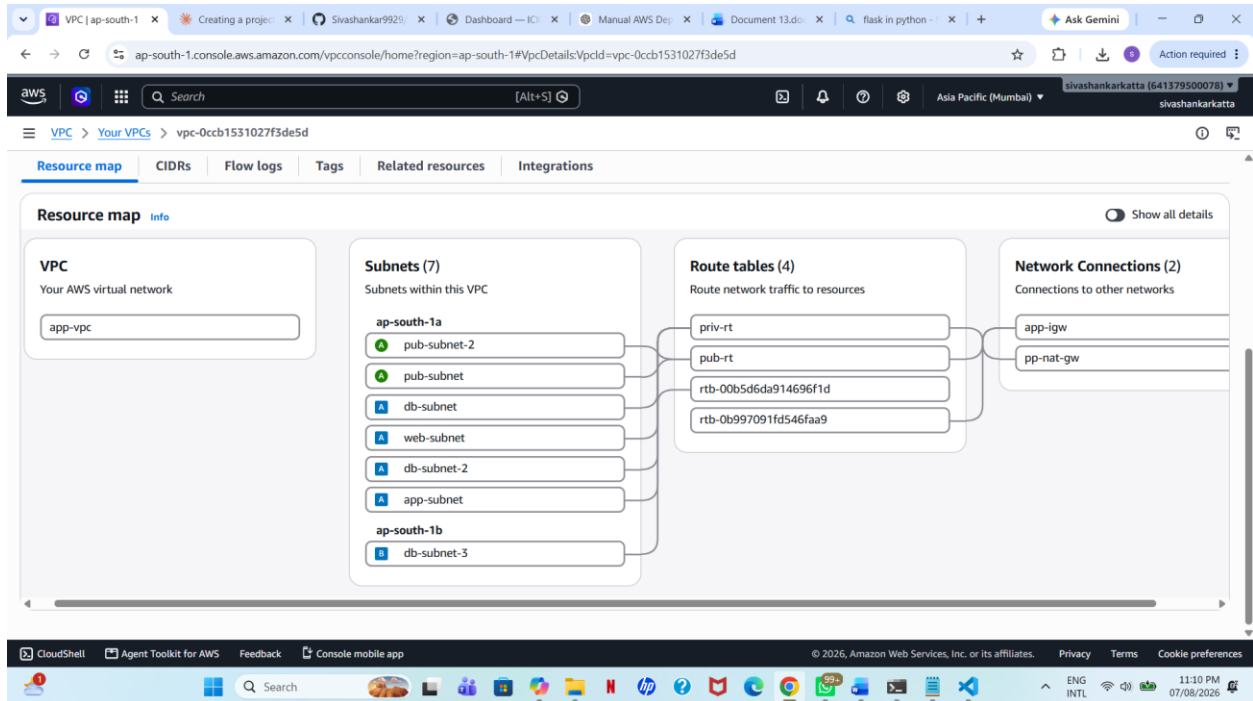


Fig. 2 — VPC Resource Map showing the manually configured AWS network

SCREENSHOT PLACEMENT

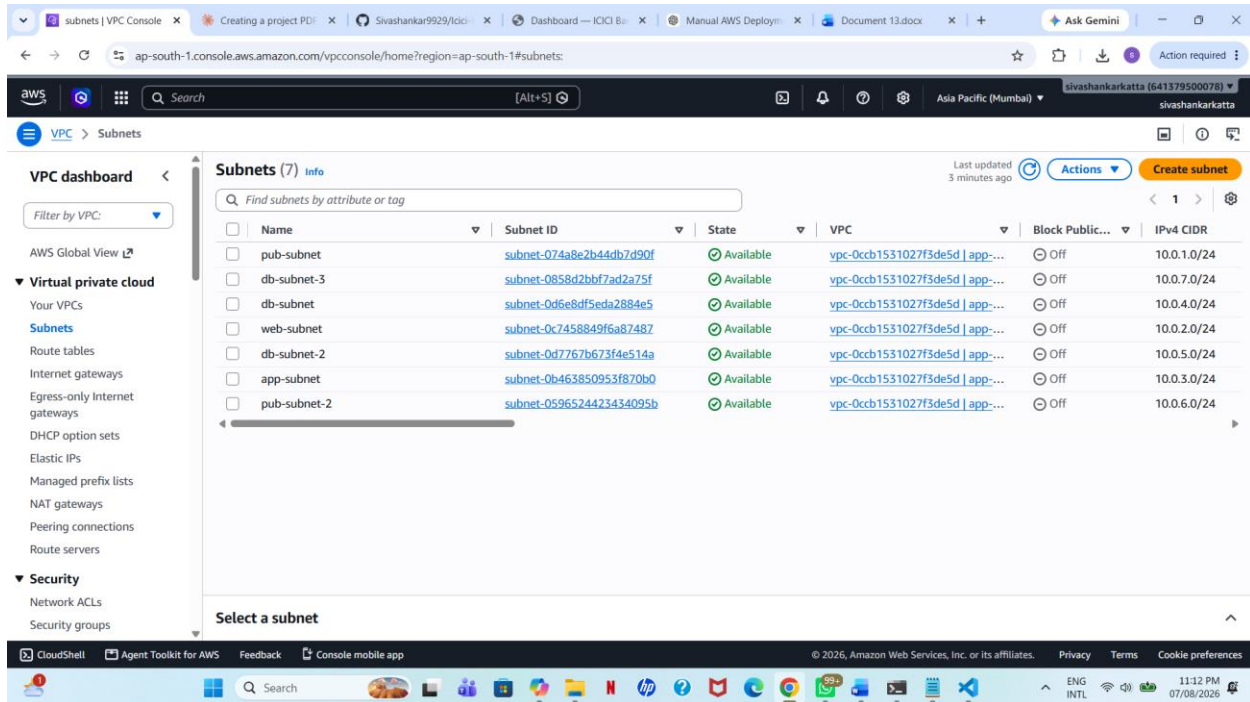


Fig. 3 — Public and private subnets manually created in the VPC

5.2 Security Group Configuration

Separate Security Groups were manually created for each tier.

- ALB Security Group
- Bastion Security Group
- Web Security Group
- Application Security Group
- Database Security Group

SCREENSHOT PLACEMENT

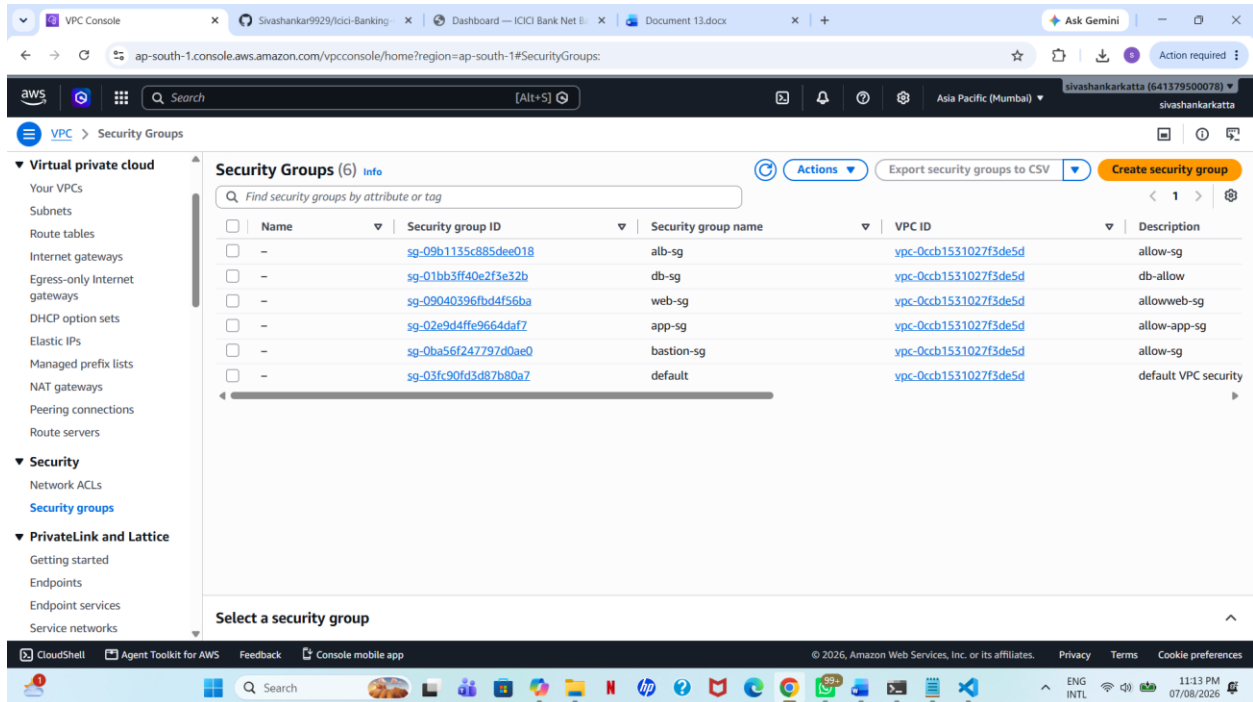
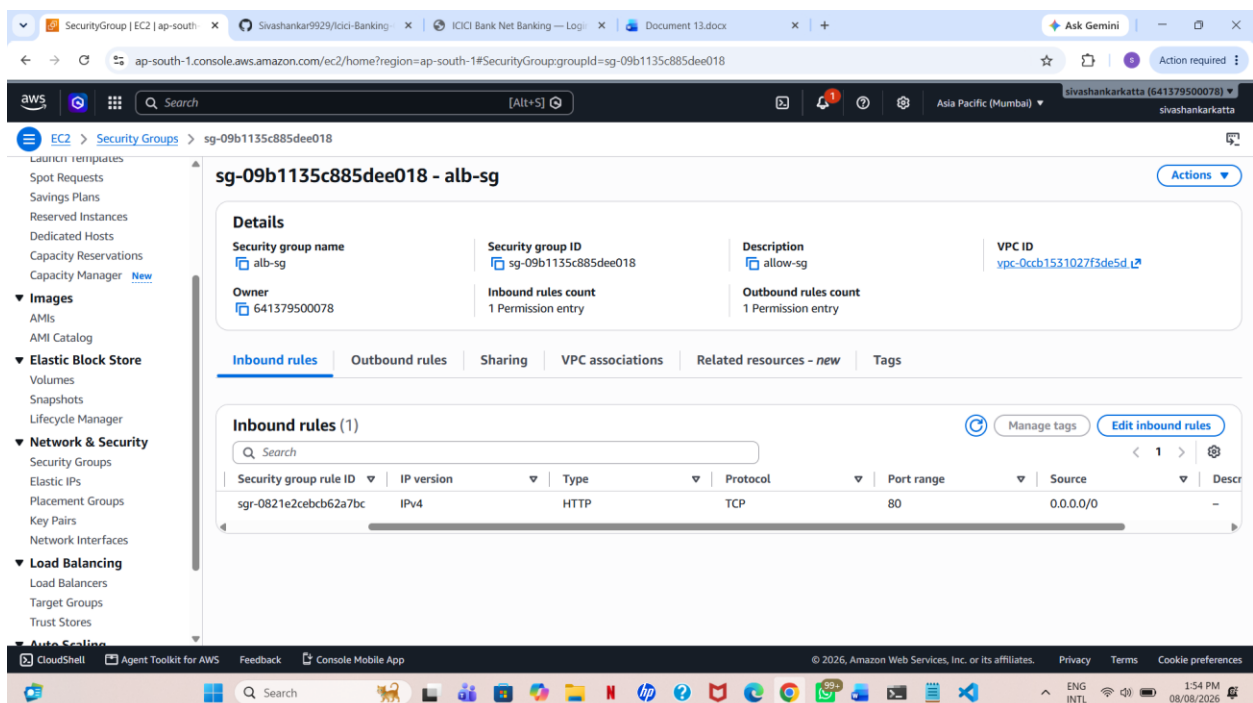


Fig. 4 — Security Groups manually configured for the 3-tier architecture

SCREENSHOT PLACEMENT



SecurityGroup | EC2 | ap-south-1 | Sivashankar9929/icici-Banking | ICICI Bank Net Banking — Log | Document 13.docx | Ask Gemini | Action required

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#SecurityGroup:groupId=sg-01bb3ff40e2f3e32b

aws [Search] [Alt+S] Asia Pacific (Mumbai) sivashankarkatta (641379500078) sivashankarkatta

EC2 > Security Groups > sg-01bb3ff40e2f3e32b

sg-01bb3ff40e2f3e32b - db-sg

Actions

Details

Security group name db-sg	Security group ID sg-01bb3ff40e2f3e32b	Description db-allow	VPC ID vpc-0ccb153102f3de5d
Owner 641379500078	Inbound rules count 1 Permission entry	Outbound rules count 1 Permission entry	

[Inbound rules](#) | [Outbound rules](#) | [Sharing](#) | [VPC associations](#) | [Related resources - new](#) | [Tags](#)

Inbound rules (1) [Manage tags](#) [Edit inbound rules](#)

Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
sgr-08bb55aac8dad2002	-	MYSQL/Aurora	TCP	3306	sg-02e9d4ffe9664daf7...	-

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences ENG INTL 1:54 PM 08/08/2026

SecurityGroup | EC2 | ap-south-1 | Sivashankar9929/icici-Banking | ICICI Bank Net Banking — Log | Document 13.docx | Ask Gemini | Action required

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#SecurityGroup:groupId=sg-09040396fbd4f56ba

aws [Search] [Alt+S] Asia Pacific (Mumbai) sivashankarkatta (641379500078) sivashankarkatta

EC2 > Security Groups > sg-09040396fbd4f56ba

sg-09040396fbd4f56ba - web-sg

Actions

Details

Security group name web-sg	Security group ID sg-09040396fbd4f56ba	Description allowweb-sg	VPC ID vpc-0ccb153102f3de5d
Owner 641379500078	Inbound rules count 3 Permission entries	Outbound rules count 1 Permission entry	

[Inbound rules](#) | [Outbound rules](#) | [Sharing](#) | [VPC associations](#) | [Related resources - new](#) | [Tags](#)

Inbound rules (3) [Manage tags](#) [Edit inbound rules](#)

Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
sgr-0147700524c45e95c	-	HTTP	TCP	80	sg-09b1135c885dee01...	-
sgr-0512fc07cbad4d3c6	-	SSH	TCP	22	sg-0ba56f247797d0ae0...	-
sgr-0cded6b9ad66784aa	-	Custom TCP	TCP	8000	sg-09040396fbd4f56ba...	-

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences ENG INTL 1:55 PM 08/08/2026

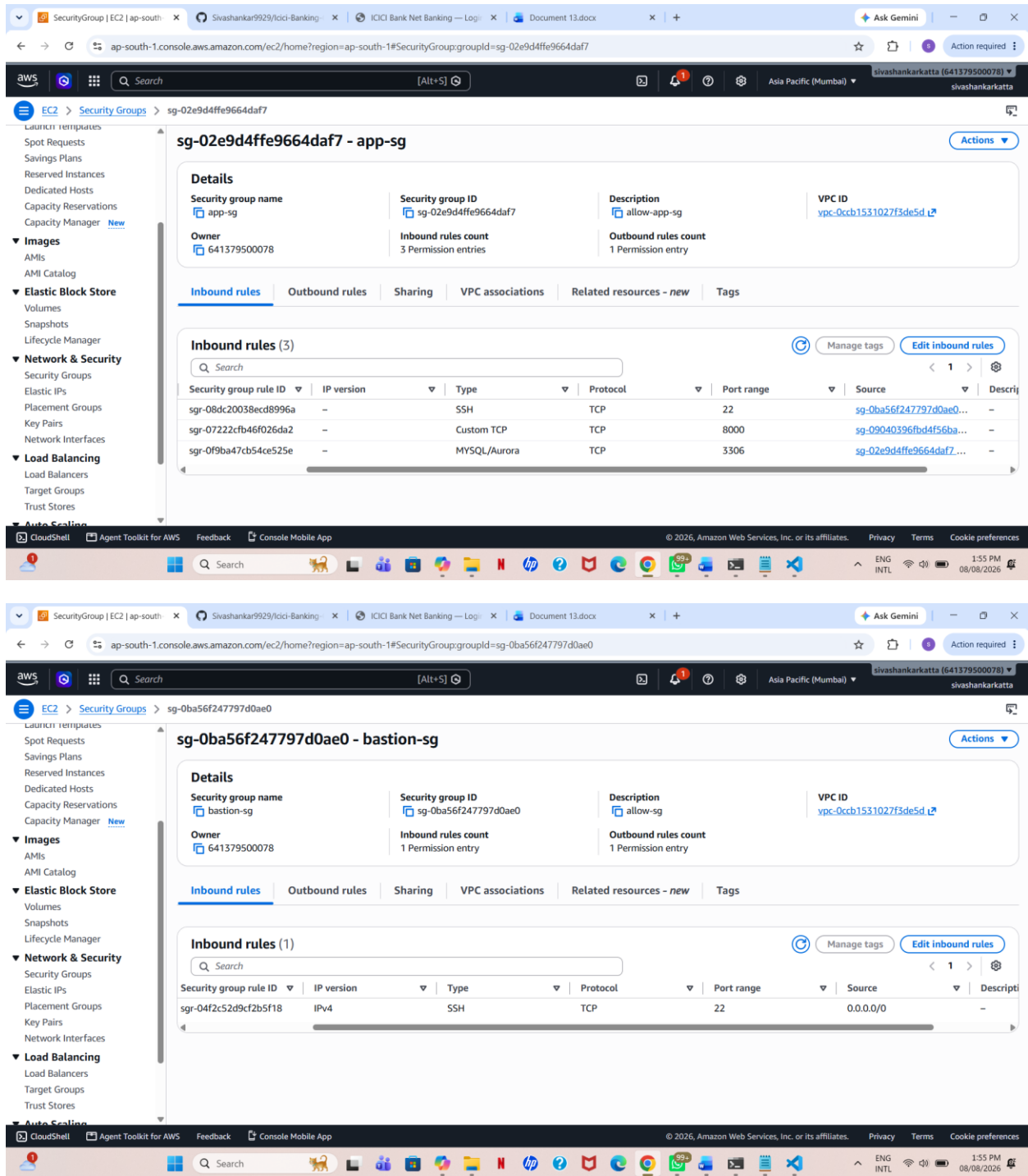


Fig. 5 — Security Group inbound rules controlling communication between application tiers

6. Manual Implementation Phases

Phase 1 – VPC Creation

The VPC was manually created using the AWS Management Console with the CIDR block:

10.0.0.0/16

DNS hostnames were enabled for the VPC.

SCREENSHOT PLACEMENT

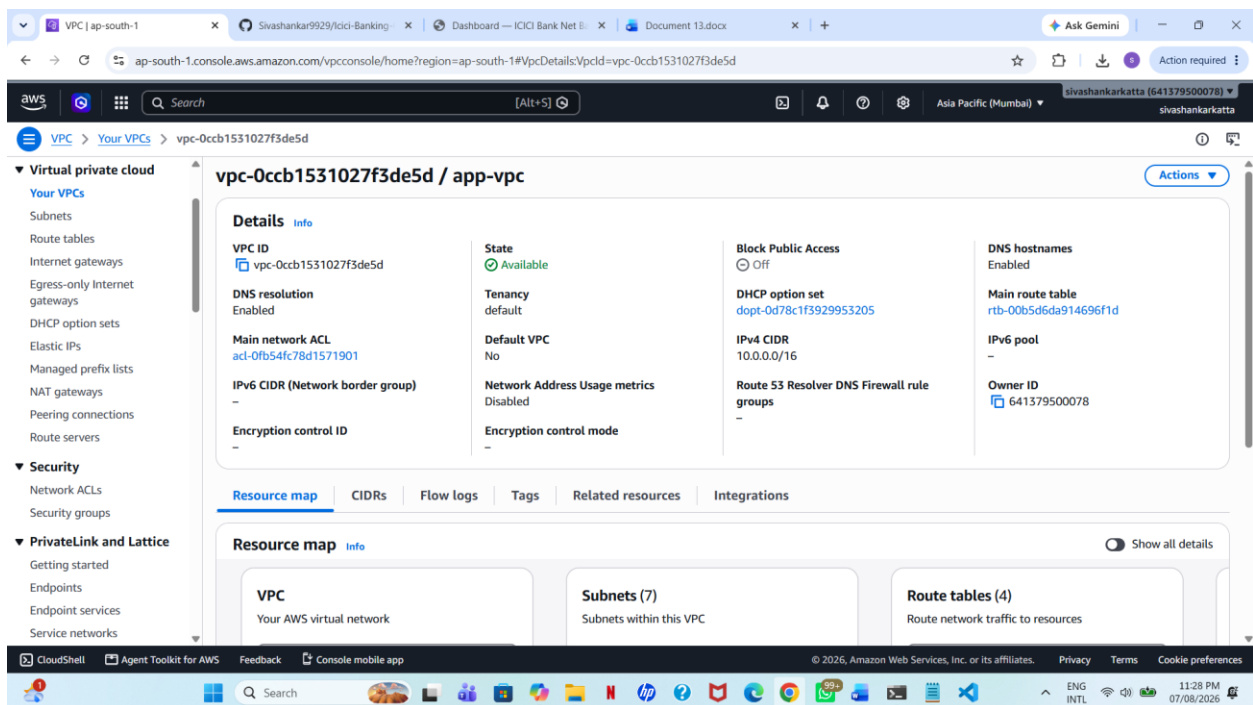


Fig. 6 — VPC manually created through the AWS Management Console

Phase 2 – Subnet Creation

Six subnets were manually created across the required Availability Zones.

The subnets were divided into public and private networks according to the application architecture.

SCREENSHOT PLACEMENT

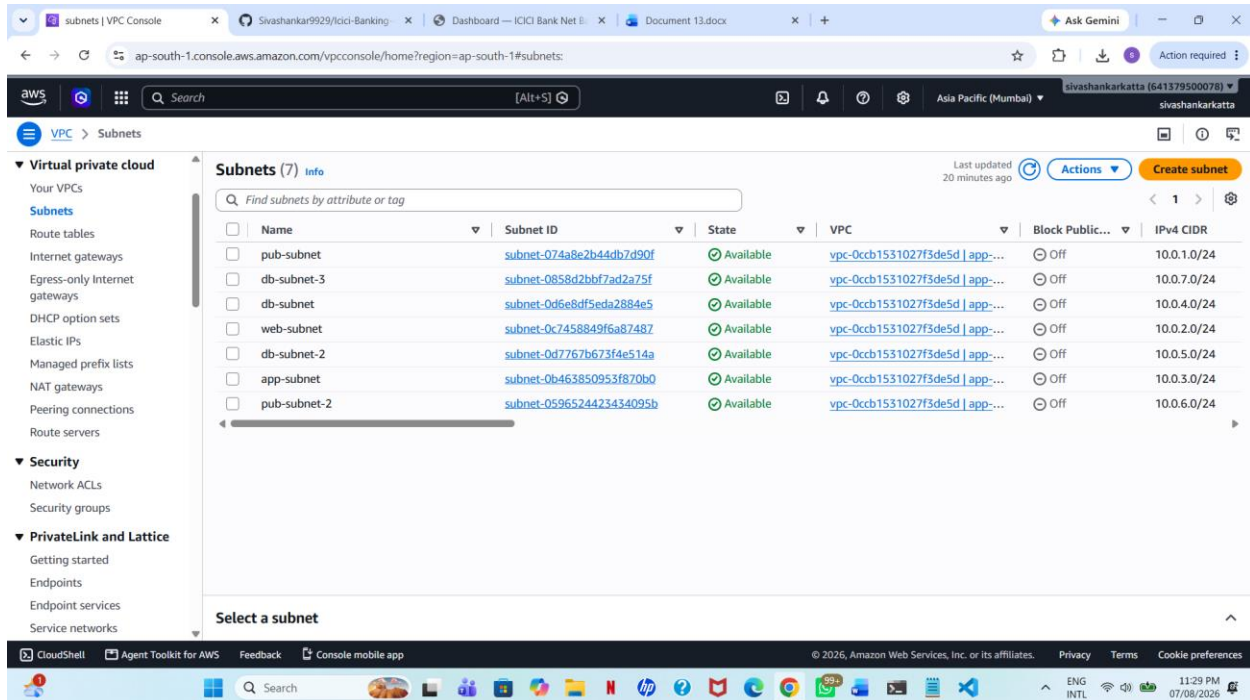


Fig. 7 — Subnets manually created across the configured Availability Zones

Phase 3 – Internet Gateway and NAT Gateway

An Internet Gateway was manually created and attached to the VPC.

A NAT Gateway was also configured to provide outbound internet connectivity for resources located in private subnets.

SCREENSHOT PLACEMENT

The figure consists of two screenshots of the AWS Management Console, showing the configuration of an Internet Gateway and a NAT Gateway for a VPC.

Top Screenshot: Internet Gateway Configuration

The top screenshot shows the AWS Management Console interface for the VPC dashboard. The left sidebar lists various VPC resources, including Internet gateways. The main content area displays the details for the Internet Gateway **igw-0fc2320adea8b2ecc**, which is associated with the VPC **app-igw**. The details section shows the Internet gateway ID, State (Attached), VPC ID, and Owner. The Tags section shows a single tag with the key **Name** and value **app-igw**.

Bottom Screenshot: NAT Gateway Configuration

The bottom screenshot shows the AWS Management Console interface for the VPC dashboard. The left sidebar lists various VPC resources, including NAT gateways. The main content area displays the details for the NAT Gateway **nat-19607d9cfd10b0640**, which is associated with the VPC **pp-nat-gw**. The details section shows the NAT gateway ID, Availability mode (Regional), State (Available), NAT gateway ARN, Connectivity type (Public), Method of EIP allocation (Automatic), and Created date (Thursday, 6 August 2026 at 14:29:10 GMT+5:30). The IP addresses section shows two associated IP addresses:

IP address	Status	Availability Zone	Allocation ID	Ass
3.7.210.4	Succeeded	aps1-az1 (ap-south-1a)	eipalloc-0184d04e92ac4c6cd	eipa
13.203.111.53	Succeeded	aps1-az3 (ap-south-1b)	eipalloc-011fd6c0aa9c1bfd	eipa

Fig. 8 — Internet Gateway and NAT Gateway manually configured for the VPC

SCREENSHOT PLACEMENT

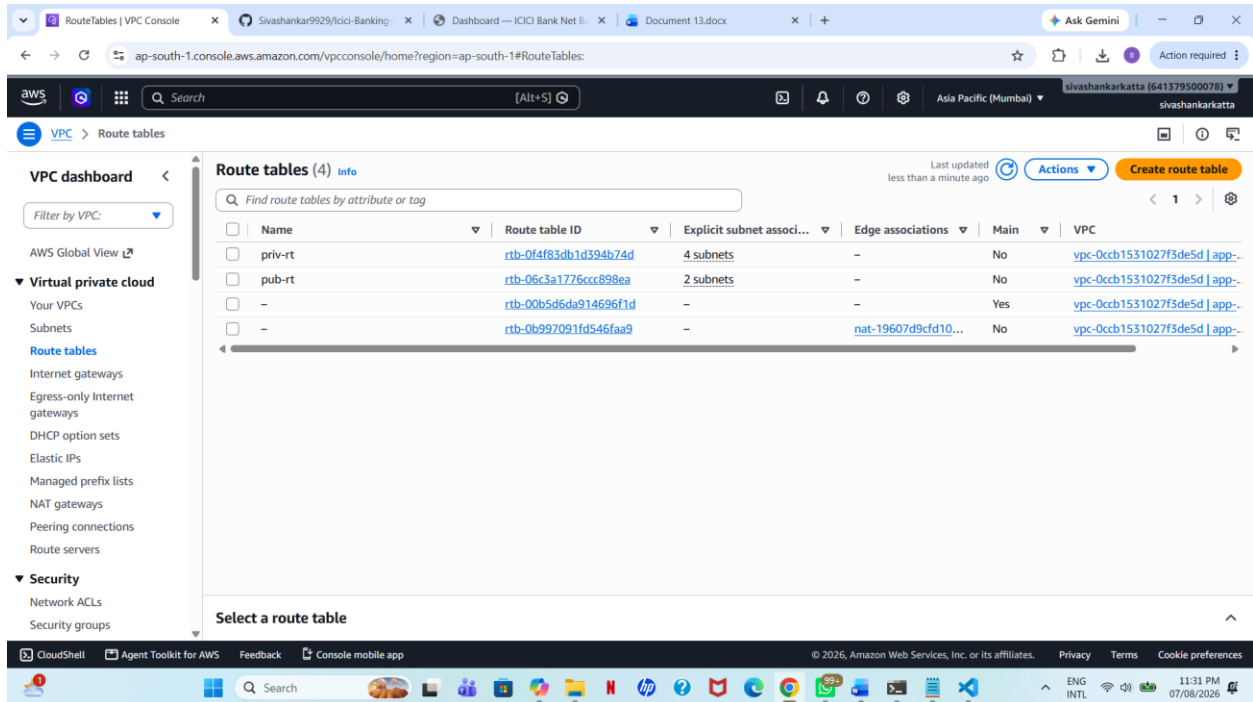


Fig. 9 — Public and private route tables manually configured

Phase 4 – Security Group Configuration

Security Groups were manually created for the ALB, Bastion Host, Web Tier, Application Tier and Database Tier.

Inbound rules were configured according to the required traffic flow between the tiers.

SCREENSHOT PLACEMENT

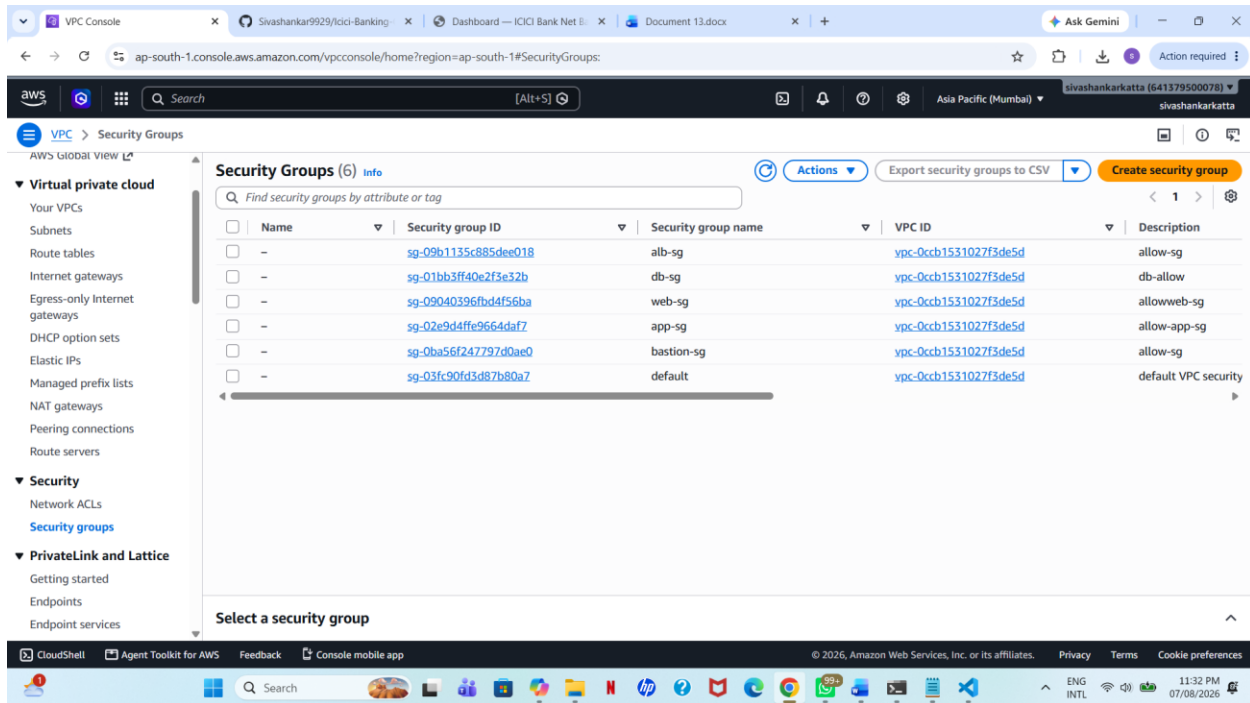


Fig. 10 — Layered Security Group rules configured for each application tier

Phase 5 – RDS MySQL Creation

Amazon RDS MySQL was manually provisioned through the AWS Management Console.

The database was configured inside the private network and public access was disabled.

SCREENSHOT PLACEMENT

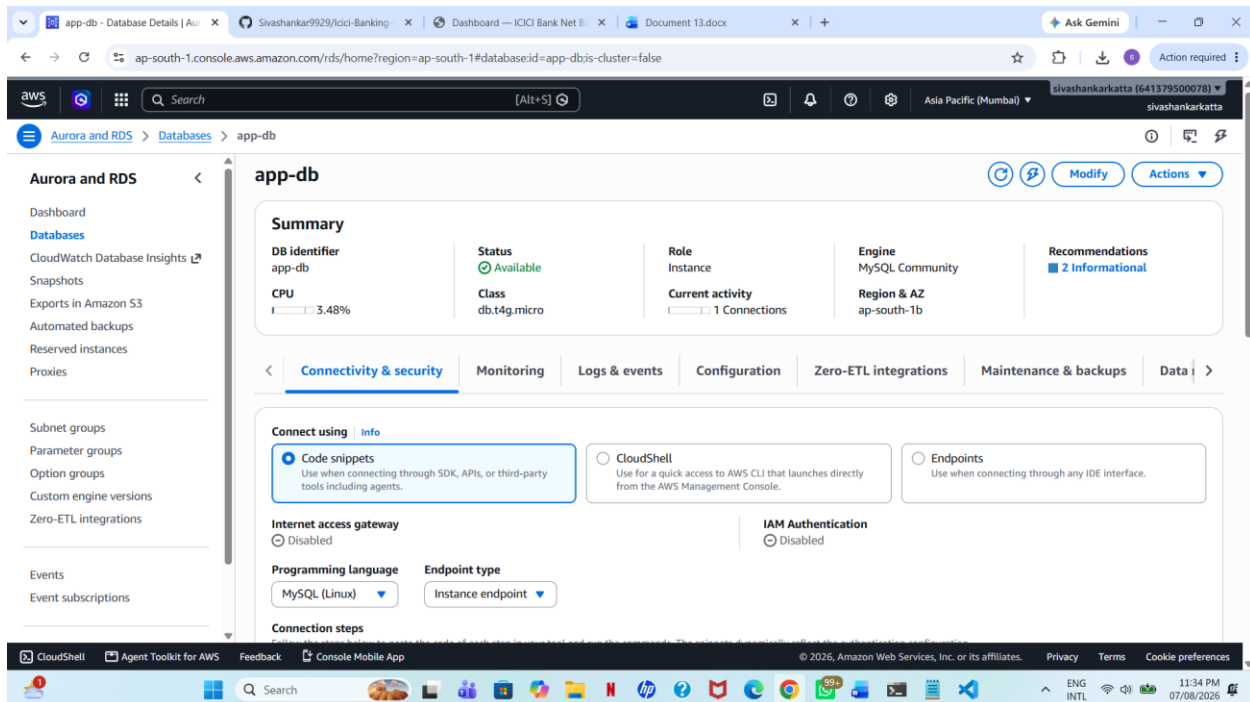


Fig. 11 — Amazon RDS MySQL database manually provisioned through AWS Console

Phase 6 – PEM Key Pair Creation and Secure SSH Access

An EC2 key pair was manually created through the AWS Management Console.

The private key was downloaded in .pem format and securely stored on the administrator's system.

The PEM private key was used for SSH authentication when accessing the EC2 instances.

Because the Web and Application servers were placed in private subnets, they were not accessed directly from the internet. Instead, the Bastion Host was used as the secure entry point for accessing the private servers.

Screenshot Placement

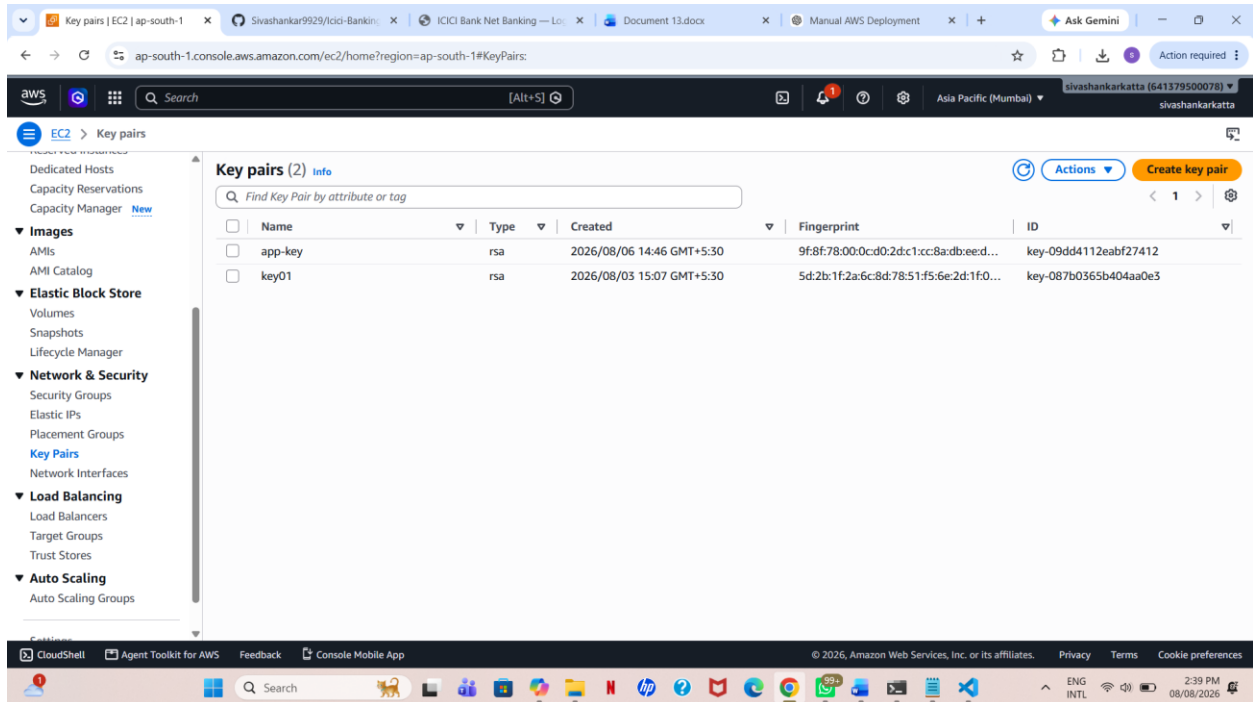


Fig. 6 — EC2 key pair created for secure SSH access

Phase 6 – EC2 Instance Creation

The required EC2 instances were manually launched using Ubuntu Server 24.04 LTS.

The instances were configured for:

- Bastion Host
- Web Server
- Application Server

SCREENSHOT PLACEMENT

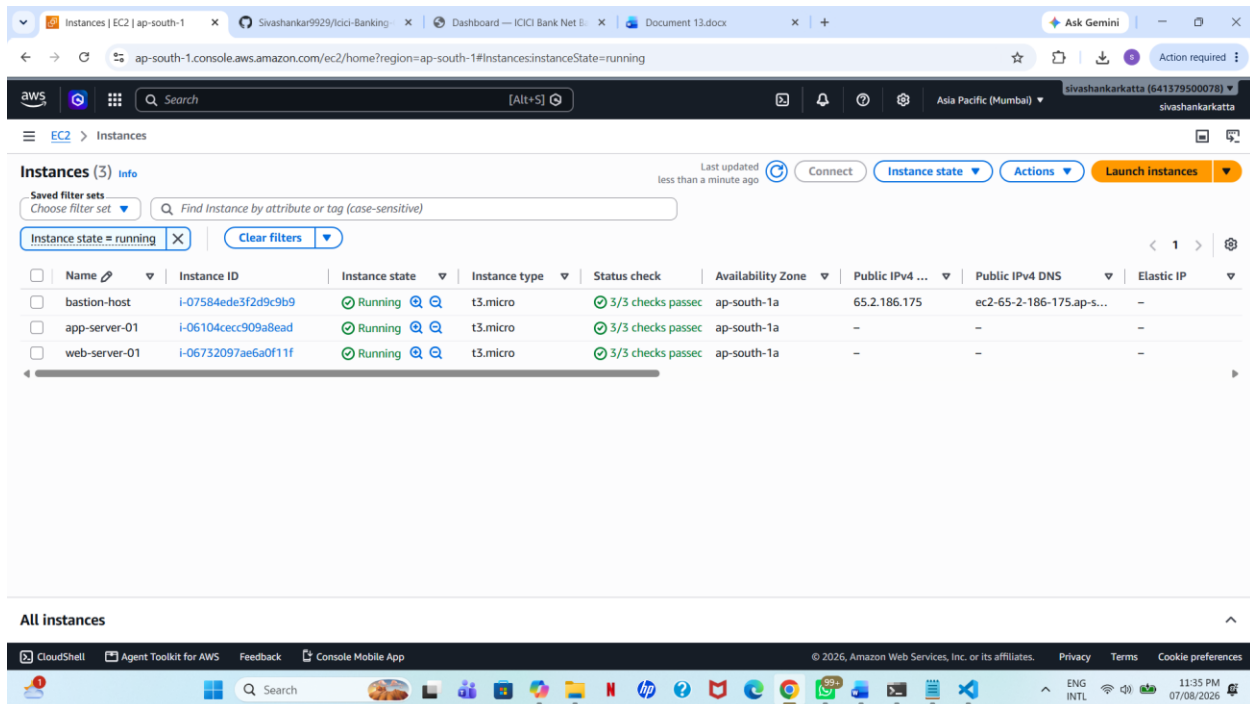


Fig. 12 — EC2 instances manually launched for the Bastion, Web and Application tiers

Screenshot Placement

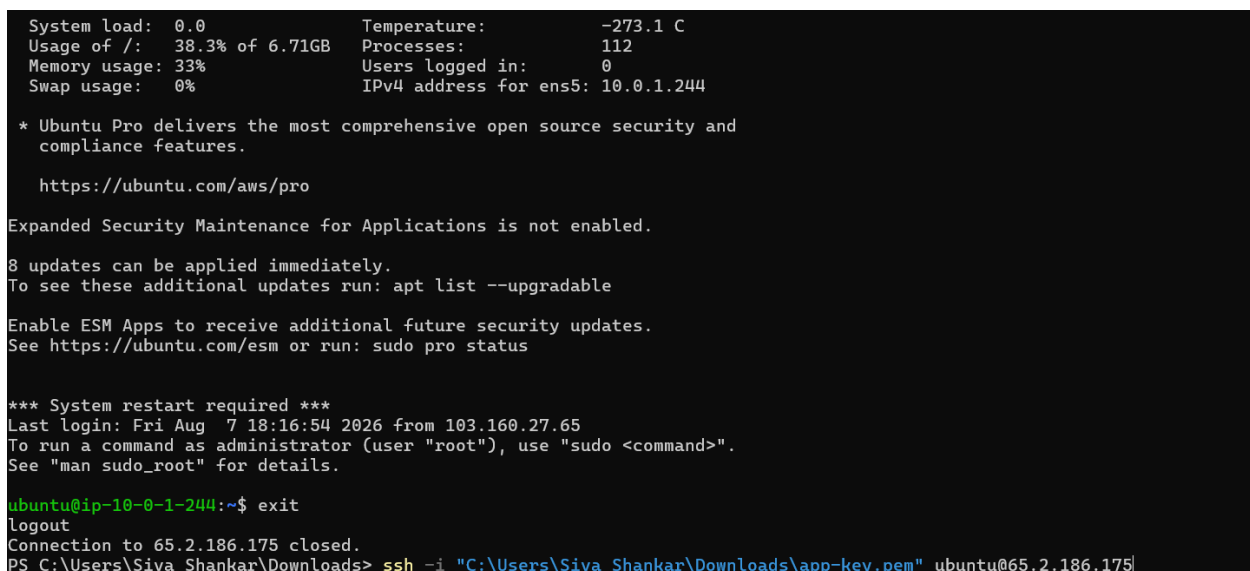


Fig. 7 — Secure SSH access to the private EC2 server through the Bastion Host

Phase 7 – Python Installation on the Application Server

After securely accessing the Application Server through the Bastion Host, Python was installed manually.

Python is required to run the Flask-based banking application.

The Python installation was verified after installation to ensure that the required Python environment was available for application deployment.

The required Python package management tools were also prepared for installing the application dependencies.

Screenshot Placement

```
* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Sat Aug  8 09:14:16 UTC 2026

System load:  0.0               Temperature:   -273.1 C
Usage of /:   44.8% of 6.71GB   Processes:    117
Memory usage: 49%              Users logged in: 0
Swap usage:   0%               IPv4 address for ens5: 10.0.3.33

* Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.
  https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

3 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Fri Aug  7 16:24:41 2026 from 10.0.1.244
ubuntu@ip-10-0-3-33:~$ python3 --version
Python 3.12.3
ubuntu@ip-10-0-3-33:~$ |
```

Fig. 8 — Python installed and verified on the Application Server

Phase 7 – Application Deployment

The Flask application was manually deployed to the Application Server.

The required Python dependencies were installed, and the application was configured to run using Gunicorn.

SCREENSHOT PLACEMENT

```
System information as of Fri Aug 7 18:16:57 UTC 2026

System load: 0.0          Temperature: -273.1 C
Usage of /: 38.3% of 6.71GB Processes: 112
Memory usage: 30%        Users logged in: 0
Swap usage: 0%           IPv4 address for ens5: 10.0.2.60

* Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

12 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Fri Aug 7 15:47:13 2026 from 10.0.1.244
ubuntu@ip-10-0-2-60:~$ ls
Icici-Banking-Clone
ubuntu@ip-10-0-2-60:~$ cd Icici-Banking-Clone/
ubuntu@ip-10-0-2-60:~/Icici-Banking-Clone$ ls
'Application Tier' Procfile README.md 'Web Tier' app.py requirements.txt
ubuntu@ip-10-0-2-60:~/Icici-Banking-Clone$ |
```

Fig. 13 — Flask banking application manually deployed on the EC2 Application Server

SCREENSHOT PLACEMENT

```
ubuntu@ip-10-0-3-33:~/Icici-Banking-Clone$ ls
'Application Tier' README.md __pycache__ flask.log requirements.txt
Procfile          'Web Tier' app.py gunicorn.log venv
ubuntu@ip-10-0-3-33:~/Icici-Banking-Clone$ sudo systemctl daemon-reload
sudo systemctl restart flaskapp
sudo systemctl status flaskapp
● flaskapp.service - Gunicorn instance for Flask app
   Loaded: loaded (/etc/systemd/system/flaskapp.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-08-07 16:35:18 UTC; 27ms ago
     Main PID: 30497 (gunicorn)
       Tasks: 1 (limit: 1013)
      Memory: 3.1M (peak: 3.1M)
         CPU: 19ms
    CGroup: /system.slice/flaskapp.service
            └─30497 /home/ubuntu/Icici-Banking-Clone/venv/bin/python3 /home/ubuntu/Icici-Banking-Clone/venv/bin/gunico>

Aug 07 16:35:18 ip-10-0-3-33 systemd[1]: Started flaskapp.service - Gunicorn instance for Flask app.
```

Fig. 14 — Gunicorn service running on the Application Server

Phase 10 – Clone Application Files on Web Tier

Git was also configured on the Web Server.

The required Web Tier files and static application content were obtained from the project repository.

The files were placed in the appropriate directory on the Web Server so that NGINX could serve the required static content.

This separates the Web Tier responsibilities from the Flask Application Tier.

Screenshot Placement

```
* Management:      https://landscape.canonical.com
* Support:         https://ubuntu.com/pro

System information as of Sat Aug  8 09:16:41 UTC 2026

System load:  0.0           Temperature:      -273.1 C
Usage of /:   38.3% of 6.71GB Processes:         113
Memory usage: 30%          Users logged in:   0
Swap usage:   0%           IPv4 address for ens5: 10.0.2.60

* Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

12 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Fri Aug  7 18:35:10 2026 from 10.0.1.244
ubuntu@ip-10-0-2-60:~$ ls
Icici-Banking-Clone flaskapp.service
ubuntu@ip-10-0-2-60:~$
```

Fig. 11 — Web Tier application files cloned and prepared on the NGINX Web Server

Phase 11 – Install Application Dependencies

After cloning the application repository, the required Python dependencies were installed on the Application Server.

The application dependency configuration was used to install the packages required for running the Flask application.

The Python environment was verified before starting the application using Gunicorn.

Screenshot Placement

```
ubuntu@ip-10-0-3-33:~/Icici-Banking-Clone$ cd
ubuntu@ip-10-0-3-33:~$ ls -ld ~/Icici-Banking-Clone/venv
drwxr-xr-x 5 root root 4096 Aug  6 11:10 /home/ubuntu/Icici-Banking-Clone/venv
ubuntu@ip-10-0-3-33:~$ sudo chown -R ubuntu:ubuntu ~/Icici-Banking-Clone/venv
ubuntu@ip-10-0-3-33:~$ source ~/Icici-Banking-Clone/venv/bin/activate
(venv) ubuntu@ip-10-0-3-33:~$ pip install -r ~/Icici-Banking-Clone/requirements.txt
Requirement already satisfied: flask>=3.0 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from -r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 1)) (3.1.3)
Requirement already satisfied: gunicorn>=21.2.0 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from -r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 2)) (26.0.0)
Requirement already satisfied: waitress>=2.2.0 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from -r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 3)) (3.0.2)
Requirement already satisfied: colorama>=0.4.6 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from -r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 4)) (0.4.6)
Requirement already satisfied: blinker>=1.9.0 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from flask>=3.0->-r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: click>=8.1.3 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from flask>=3.0->-r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 1)) (8.4.2)
Requirement already satisfied: itsdangerous>=2.2.0 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from flask>=3.0->-r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 1)) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from flask>=3.0->-r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 1)) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from flask>=3.0->-r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 1)) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from flask>=3.0->-r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 1)) (3.1.8)
Requirement already satisfied: packaging in ./Icici-Banking-Clone/venv/lib/python3.12/site-packages (from gunicorn>=21.2.0->-r /home/ubuntu/Icici-Banking-Clone/requirements.txt (line 2)) (26.3)
(venv) ubuntu@ip-10-0-3-33:~$
```

Fig. 12 — Required Python application dependencies installed on the Application Server

Phase 8 – NGINX Configuration

NGINX was manually installed on the Web Server.

The NGINX configuration was created to:

- Serve static files directly.
- Forward dynamic requests to the Flask Application Server.
- Act as a reverse proxy between the Web Tier and Application Tier.

SCREENSHOT PLACEMENT

```
GNU nano 7.2 /etc/nginx/sites-available/flaskapp *
server {
    listen 80;
    server_name _;

    root /home/ubuntu/Icici-Banking-Clone/Web\ Tier/static;
    index index.html;

    location /static {
        alias /home/ubuntu/Icici-Banking-Clone/Web\ Tier/static;
    }

    location / {
        proxy_pass http://10.0.3.33:8000; # App server private IP
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy
```

Fig. 15 — NGINX reverse proxy configuration for the Flask Application

SCREENSHOT PLACEMENT

```
ubuntu@ip-10-0-2-60:~$ sudo nano /etc/nginx/sites-available/flaskapp
ubuntu@ip-10-0-2-60:~$ sudo systemctl enable nginx
sudo systemctl start nginx
sudo systemctl status nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-08-07 06:39:34 UTC; 11h ago
     Docs: man:nginx(8)
  Main PID: 12686 (nginx)
    Tasks: 3 (limit: 1013)
  Memory: 6.2M (peak: 6.3M)
     CPU: 2.163s
  CGroup: /system.slice/nginx.service
          └─12686 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
            └─32798 "nginx: worker process"
              └─32799 "nginx: worker process"

Aug 07 07:17:44 ip-10-0-2-60 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy se>
Aug 07 07:17:44 ip-10-0-2-60 nginx[32796]: 2026/08/07 07:17:44 [notice] 32796#32796: signal process started
Aug 07 07:17:44 ip-10-0-2-60 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy ser>
lines 1-16/16 (END)
```

Fig. 16 — NGINX service running on the Web Server

Phase 9 – Application Load Balancer

An Application Load Balancer was manually created through the AWS Management Console.

A target group was created and the Web Server was registered as a target.

SCREENSHOT PLACEMENT

The image displays two screenshots of the AWS Management Console, illustrating the configuration of an Application Load Balancer (app-alb).

Top Screenshot: Details Tab

- Load balancer type:** Application
- Status:** Active
- Hosted zone:** ZP97RAFLXTNZK
- VPC:** vpc-0ccb1531027f3de5d
- Load balancer IP address type:** IPv4
- Availability Zones:** subnet-074a8e2b44db7d90f (ap-south-1a (aps1-az1)), subnet-0858d2bbf7ad2a75f (ap-south-1b (aps1-az3))
- Date created:** August 7, 2026, 12:33 (UTC+05:30)
- Load balancer ARN:** arn:aws:elasticloadbalancing:ap-south-1:641379500078:loadbalancer/app/app-alb/de5f49146933db51
- DNS name:** app-alb-621285691.ap-south-1.elb.amazonaws.com (A Record)

Bottom Screenshot: Listeners and rules Tab

- Listeners and rules (1):** Info
- Filter listeners:** [Search bar]
- Listeners and rules table:**

Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS certificate
HTTP:80	Forward to target group web-tg (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable

Fig. 17 — Application Load Balancer manually configured for public application access

SCREENSHOT PLACEMENT

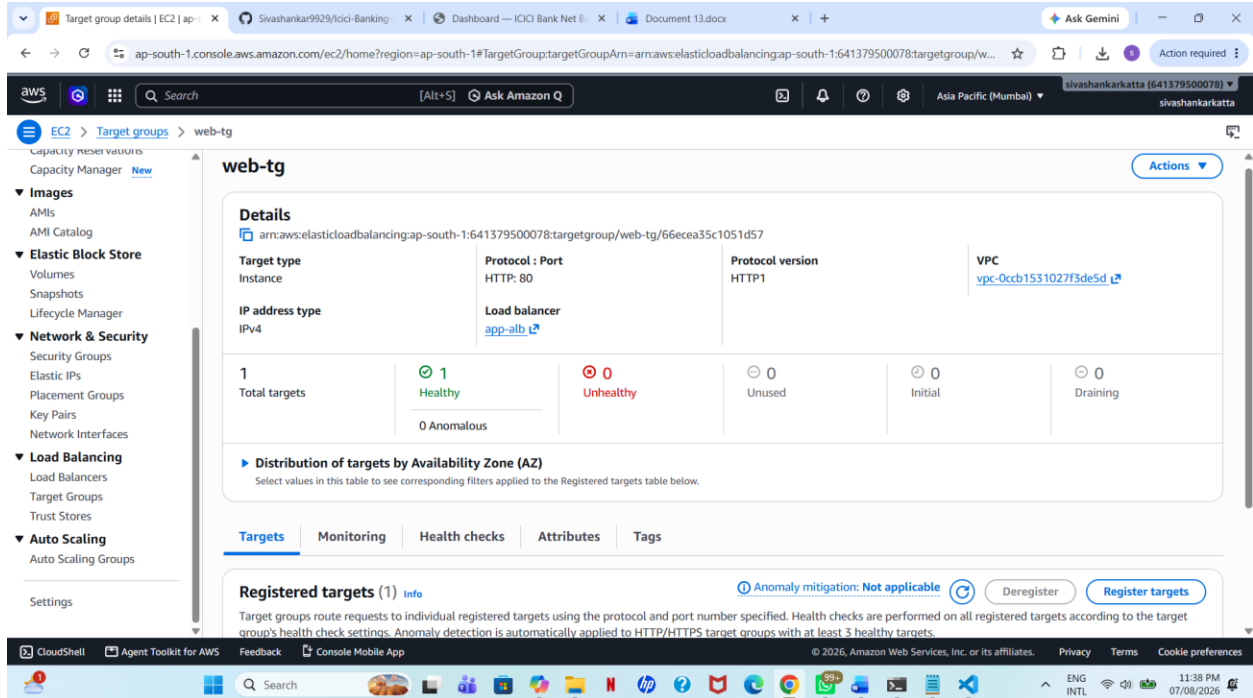


Fig. 18 — Target Group showing the Web Server in a Healthy state

Phase 10 – End-to-End Testing

The complete application was tested using the public DNS name of the Application Load Balancer.

The following application flow was verified:

Landing Page → Signup → Login → Dashboard → Logout

SCREENSHOT PLACEMENT

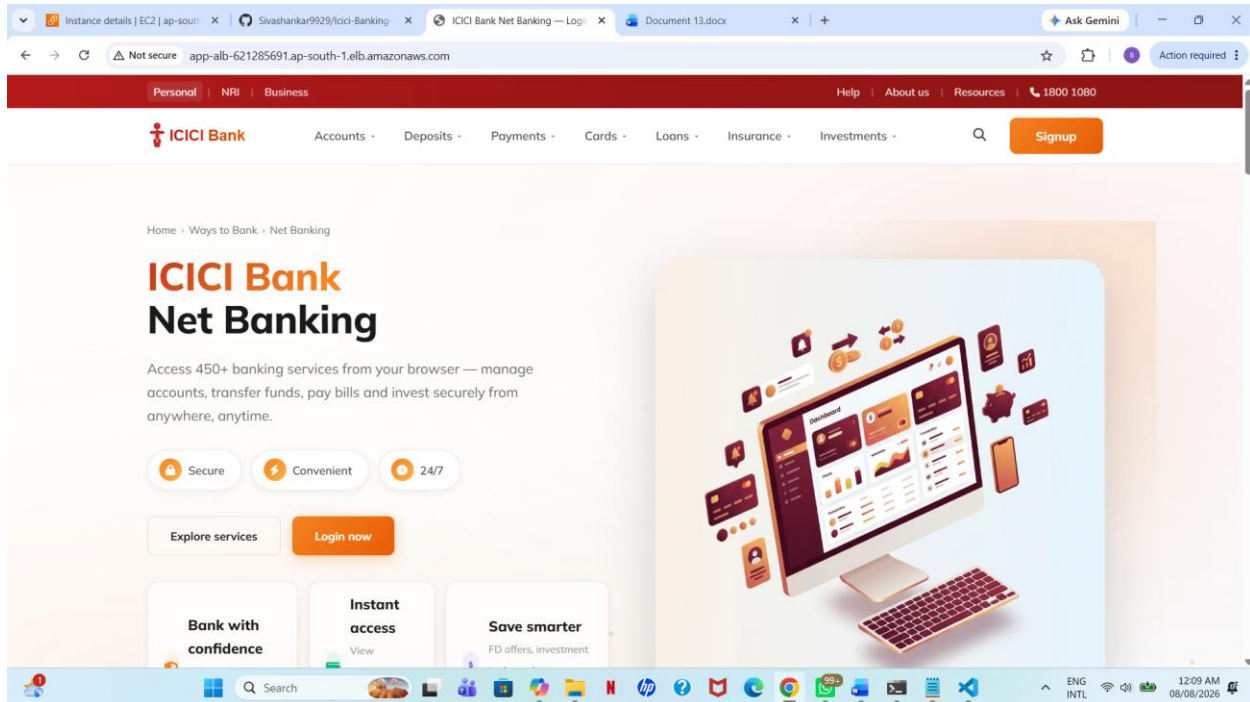


Fig. 19 — Banking application accessed through the Application Load Balancer DNS

7. Application Screenshots

7.1 Landing Page

SCREENSHOT PLACEMENT

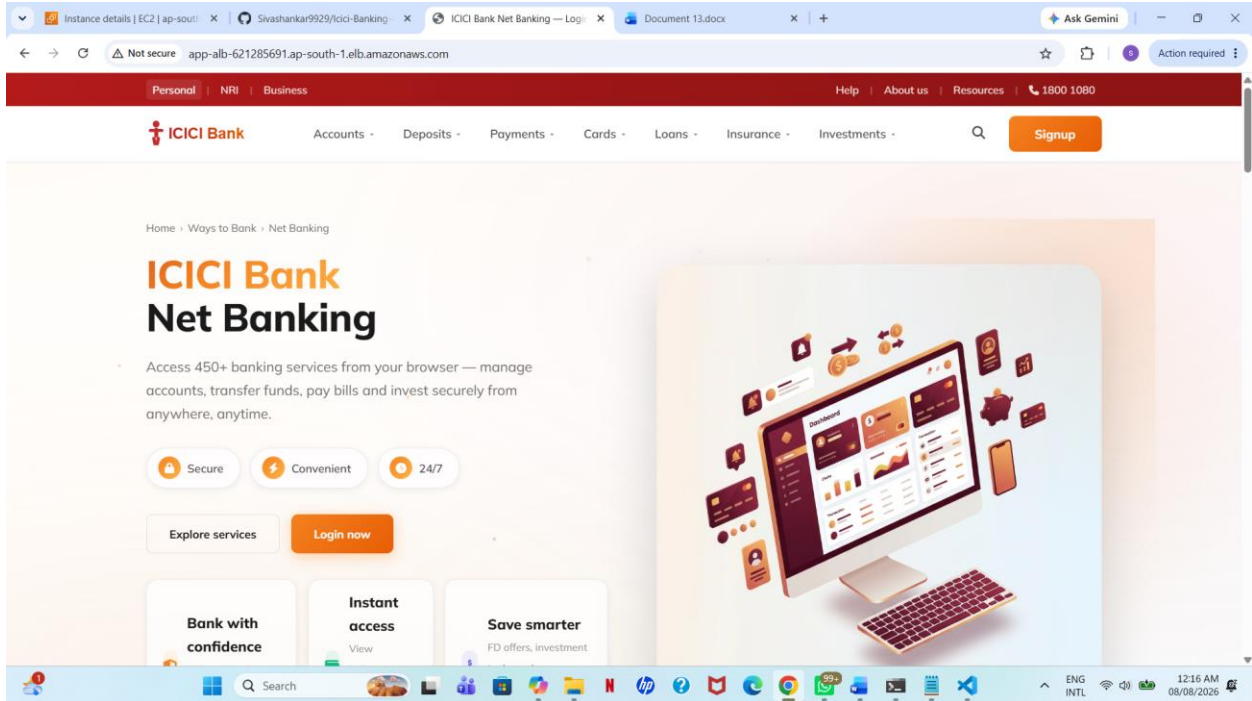
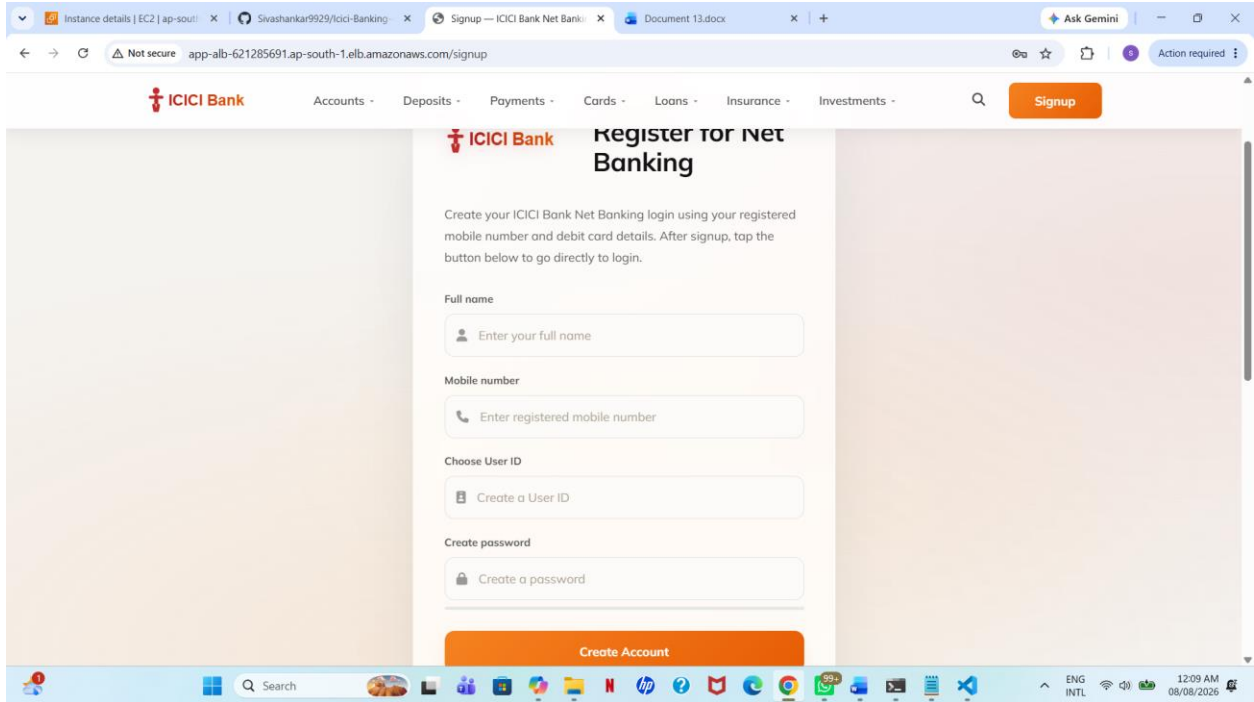


Fig. 20 — Banking application landing page

7.2 Signup Page

SCREENSHOT PLACEMENT



Instance details | EC2 | ap-sou... | Sivashankar9929/icici-Banking... | Signup - ICICI Bank Net Bank... | Document 13.docx

Not secure app-alb-621285691.ap-south-1.elb.amazonaws.com/signup

ICICI Bank Accounts - Deposits - Payments - Cards - Loans - Insurance - Investments -

Register for Net Banking

Create your ICICI Bank Net Banking login using your registered mobile number and debit card details. After signup, tap the button below to go directly to login.

Full name

Enter your full name

Mobile number

Enter registered mobile number

Choose User ID

Create a User ID

Create password

Create a password

Create Account

ENG INTL 12:09 AM 08/08/2026

Fig. 21 — User registration page

7.3 Login Page

SCREENSHOT PLACEMENT

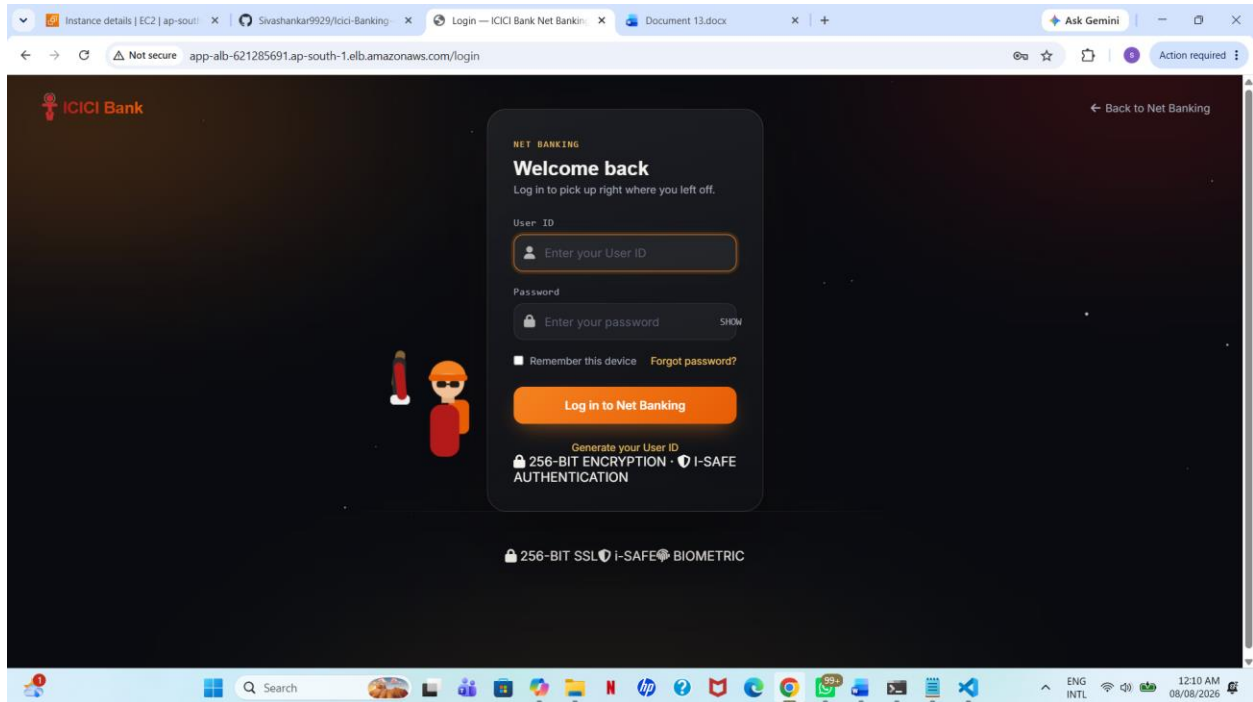


Fig. 22 — User login page

7.4 Dashboard

SCREENSHOT PLACEMENT

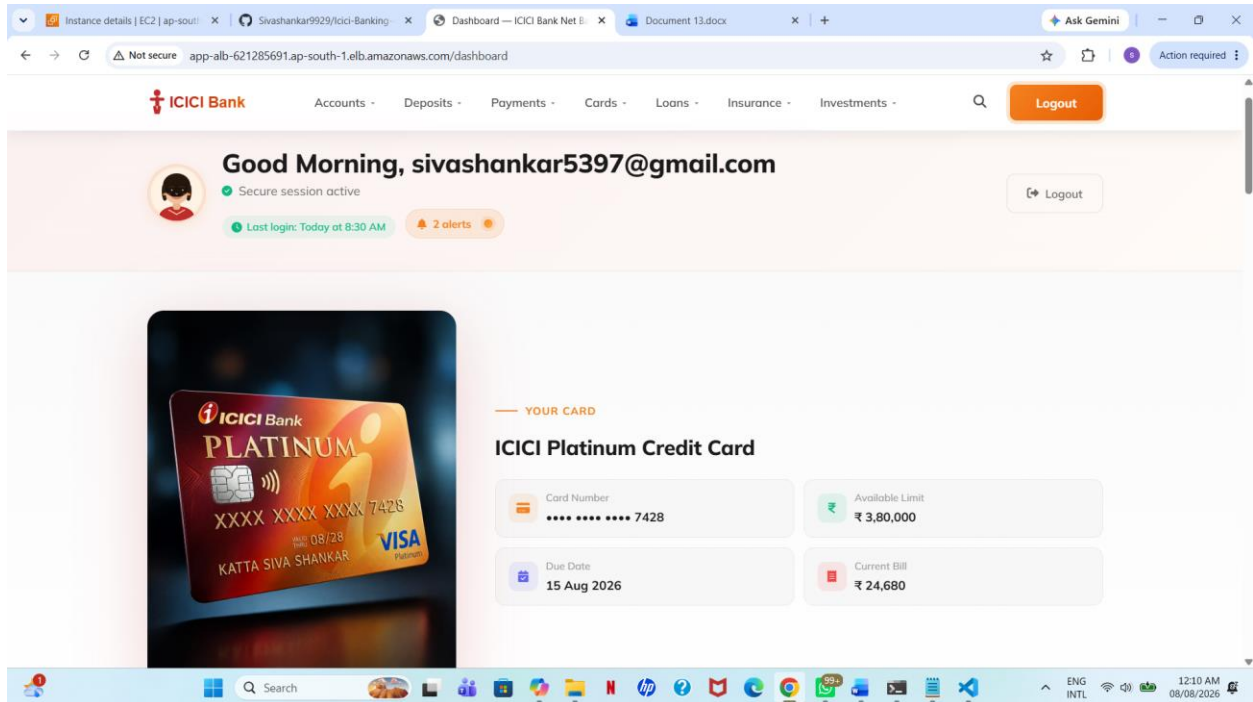


Fig. 23 — Authenticated user banking dashboard

Phase 15 – Application-to-RDS MySQL Connectivity

Amazon RDS MySQL was manually provisioned through the AWS Management Console.

The Flask Application Tier was configured to communicate with the RDS MySQL database using the database endpoint, MySQL port and required authentication details.

The Database Security Group was configured to allow MySQL traffic from the Application Tier.

The database connection was tested from the Application Server before performing application database operations.

Screenshot Placement

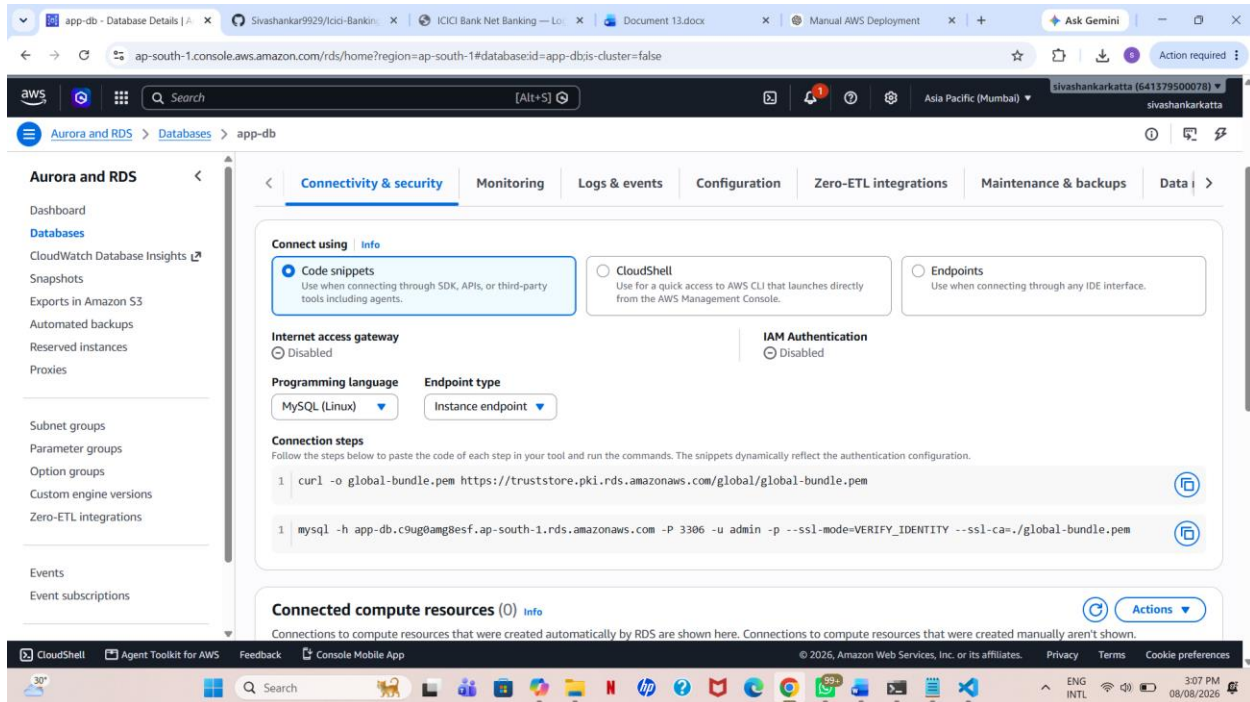


Fig. 16 — Amazon RDS MySQL database manually provisioned through AWS Console

Screenshot Placement

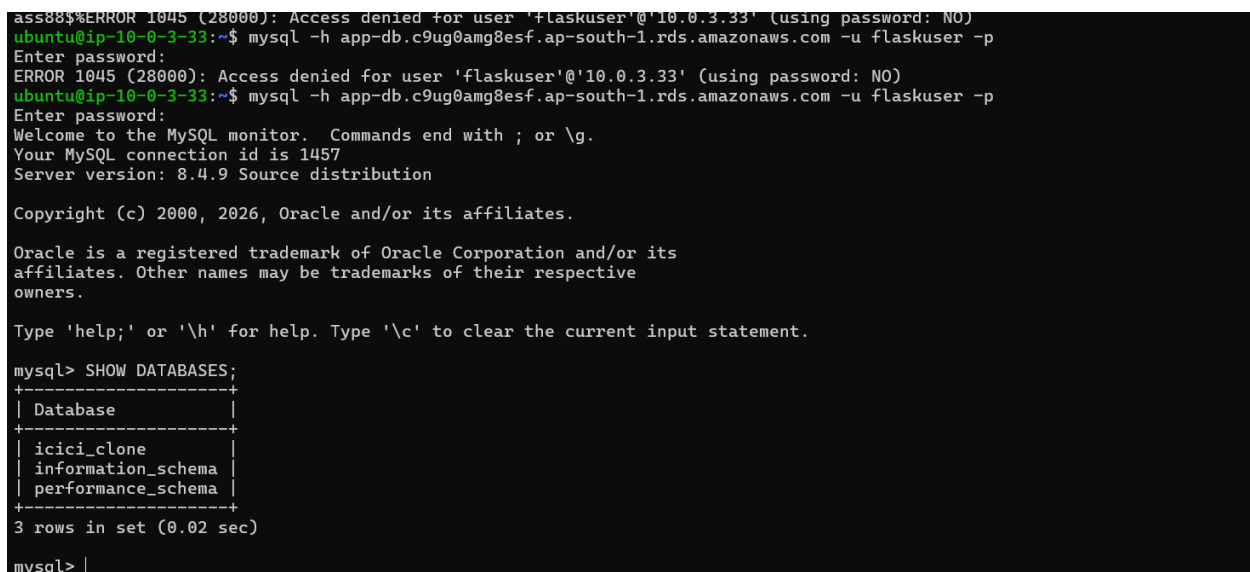


Fig. 17 — Flask Application Server successfully connected to Amazon RDS MySQL

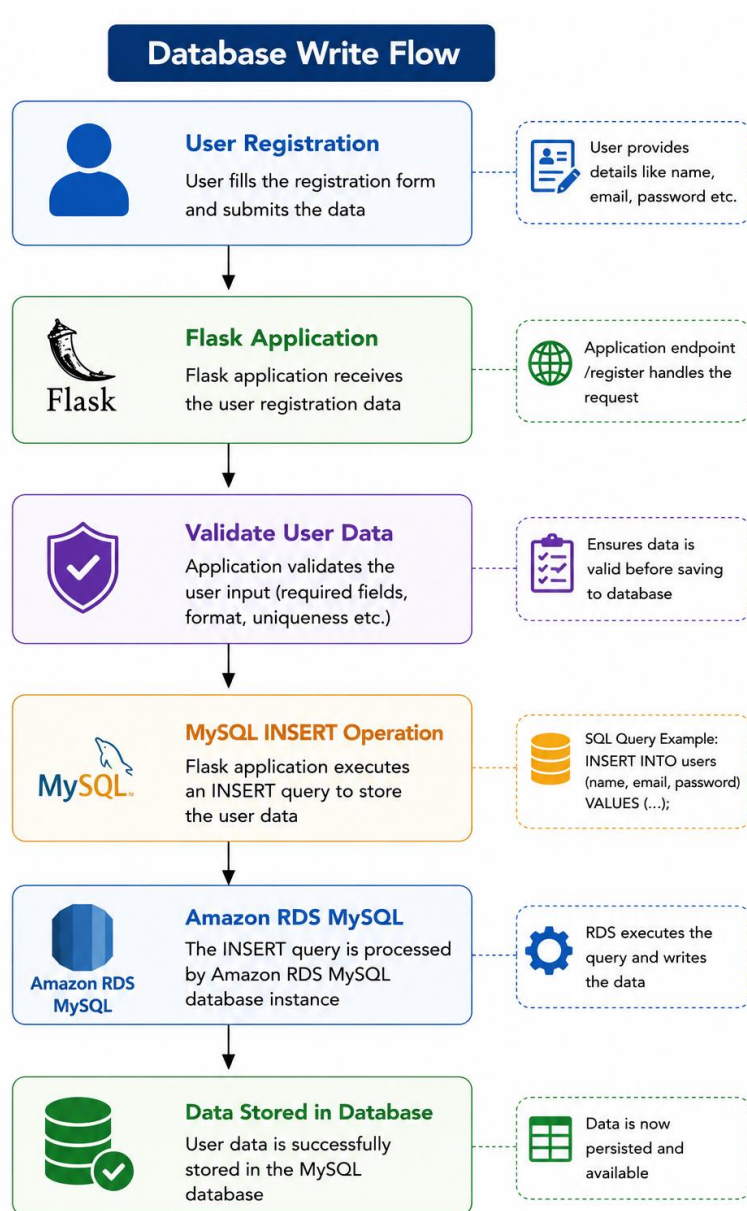
Phase 16 – Writing Application Data to Amazon RDS MySQL

After establishing connectivity between the Flask Application Tier and Amazon RDS MySQL, application data was written to the database.

When a user performs an operation such as registration, the Flask application receives the submitted information and processes the request.

The application then sends the appropriate SQL INSERT operation to Amazon RDS MySQL.

Database Write Flow



The stored data was verified in the MySQL database to confirm that the application was successfully writing data to RDS.

Screenshot Placement

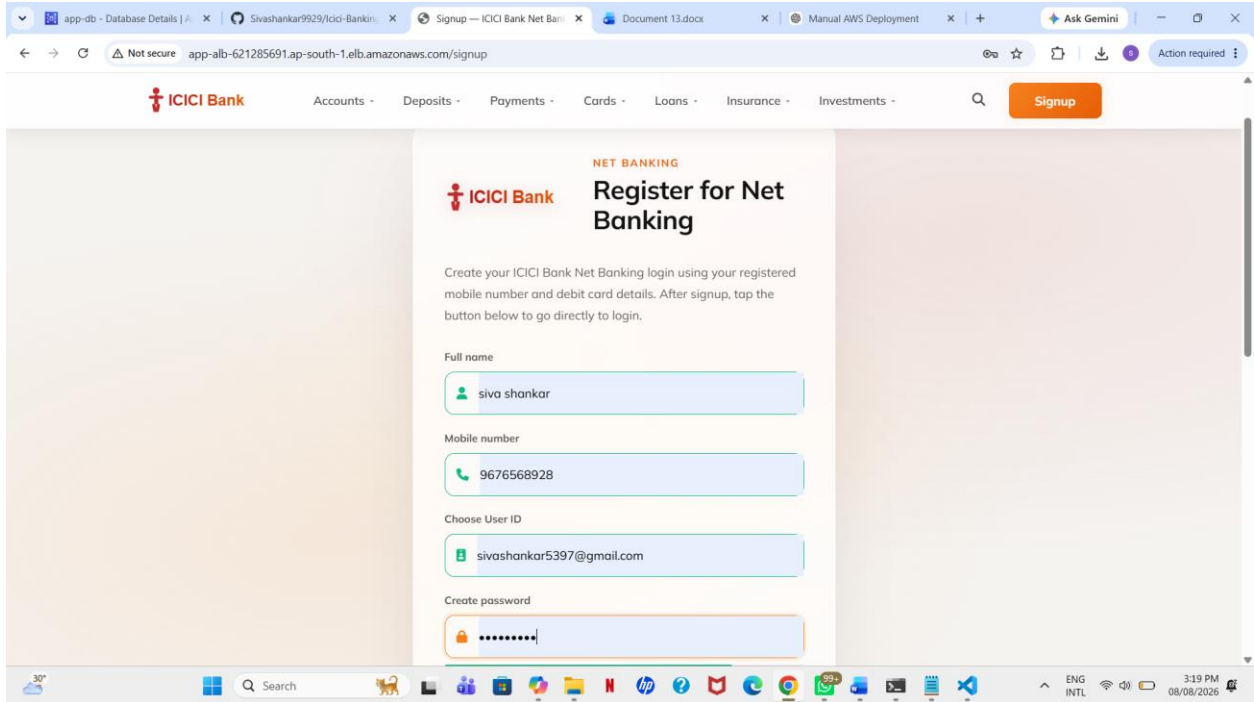


Fig. 18 — User data submitted through the banking application

Screenshot Placement

```
Database
+-----+
| icici_clone
| information_schema
| performance_schema
+-----+
3 rows in set (0.02 sec)

mysql> USE icici_clone;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM users;
+-----+-----+-----+-----+
| id | username                | password                | email                | created_at            |
+-----+-----+-----+-----+
| 1 | siva                    | hashedpassword123      | siva@example.com     | 2026-08-06 12:23:54 |
| 2 | sivashankar5397@gmail.com | sscript:32768:8:1$amCwWcdic7YrQI1y$4c6ec9908b040c007ed7fc060d71cf43ca9e2859877497ac97715da1d0f992ca247161029201bd47d307a459e2b24dd29ad52c9e8f05ff2180b94c72068ac317 | 2026-08-07 16:36:27 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

Fig. 19 — Application data successfully written to Amazon RDS MySQL

12. Conclusion

This project successfully demonstrates the manual deployment of a 3-tier banking web application on AWS.

The complete AWS infrastructure was created manually through the AWS Management Console, including the VPC, subnets, route tables, Internet Gateway, NAT Gateway, Security Groups, EC2 instances, RDS database, Target Group and Application Load Balancer.

The Flask banking application was manually deployed on an EC2 Application Server and executed using Gunicorn. NGINX was manually configured on the Web Tier to serve static content and reverse-proxy application requests.

The final architecture provides separation between the Web, Application and Database tiers while implementing network isolation and controlled communication using AWS Security Groups.

The project provides practical experience in AWS networking, EC2 administration, NGINX configuration, Flask application deployment, Amazon RDS, Load Balancing and manual cloud infrastructure management.